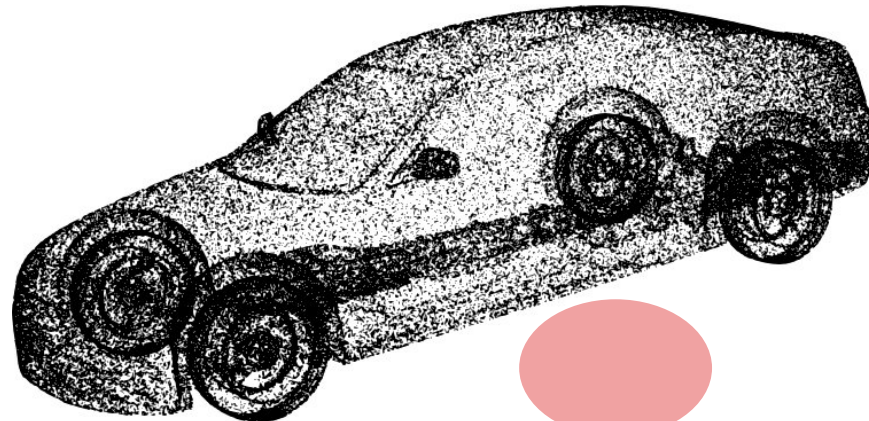
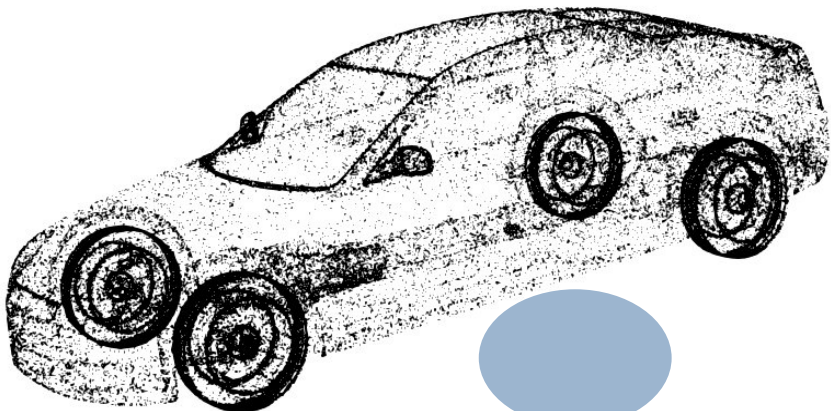
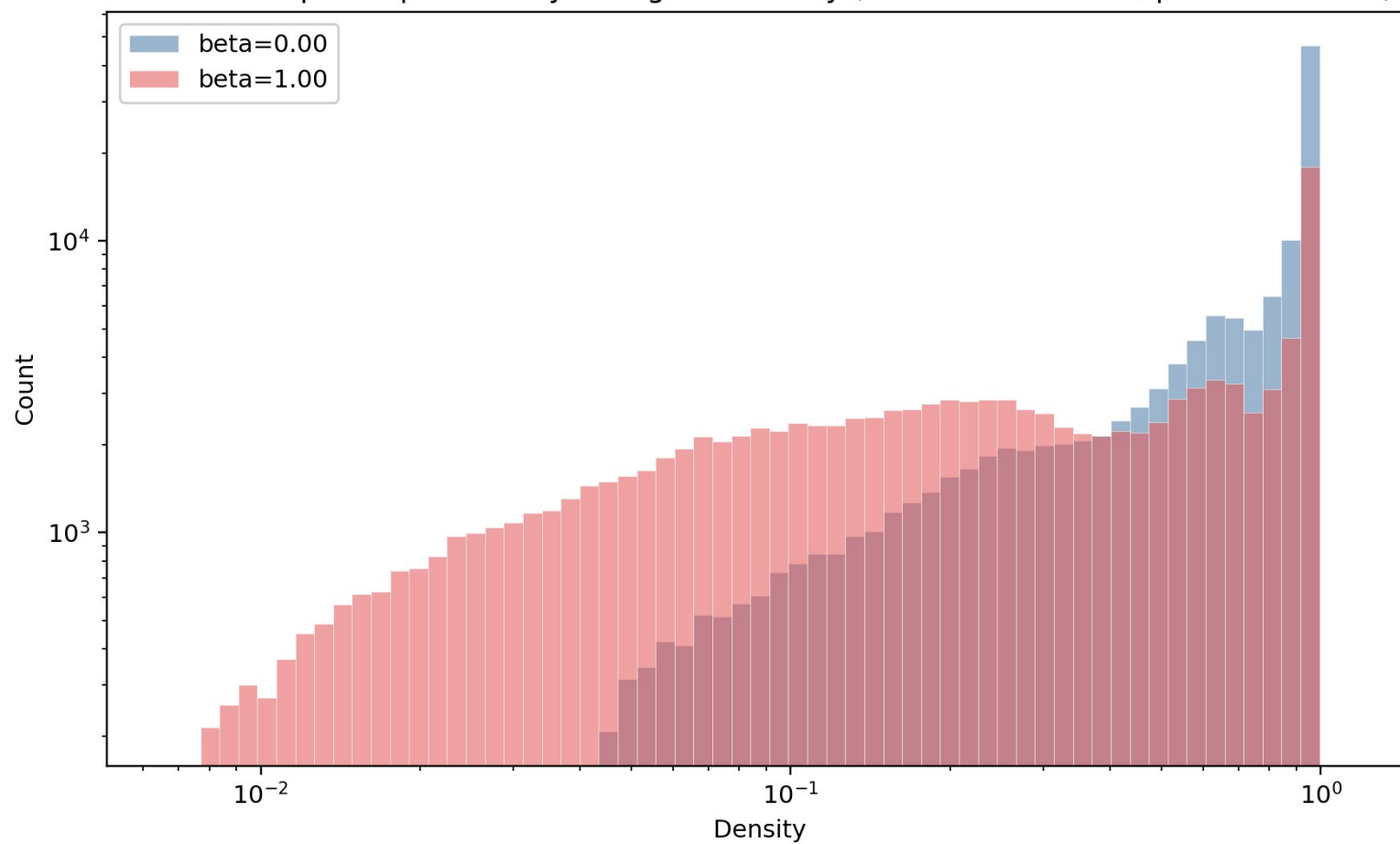


Meeting 15.06.2026

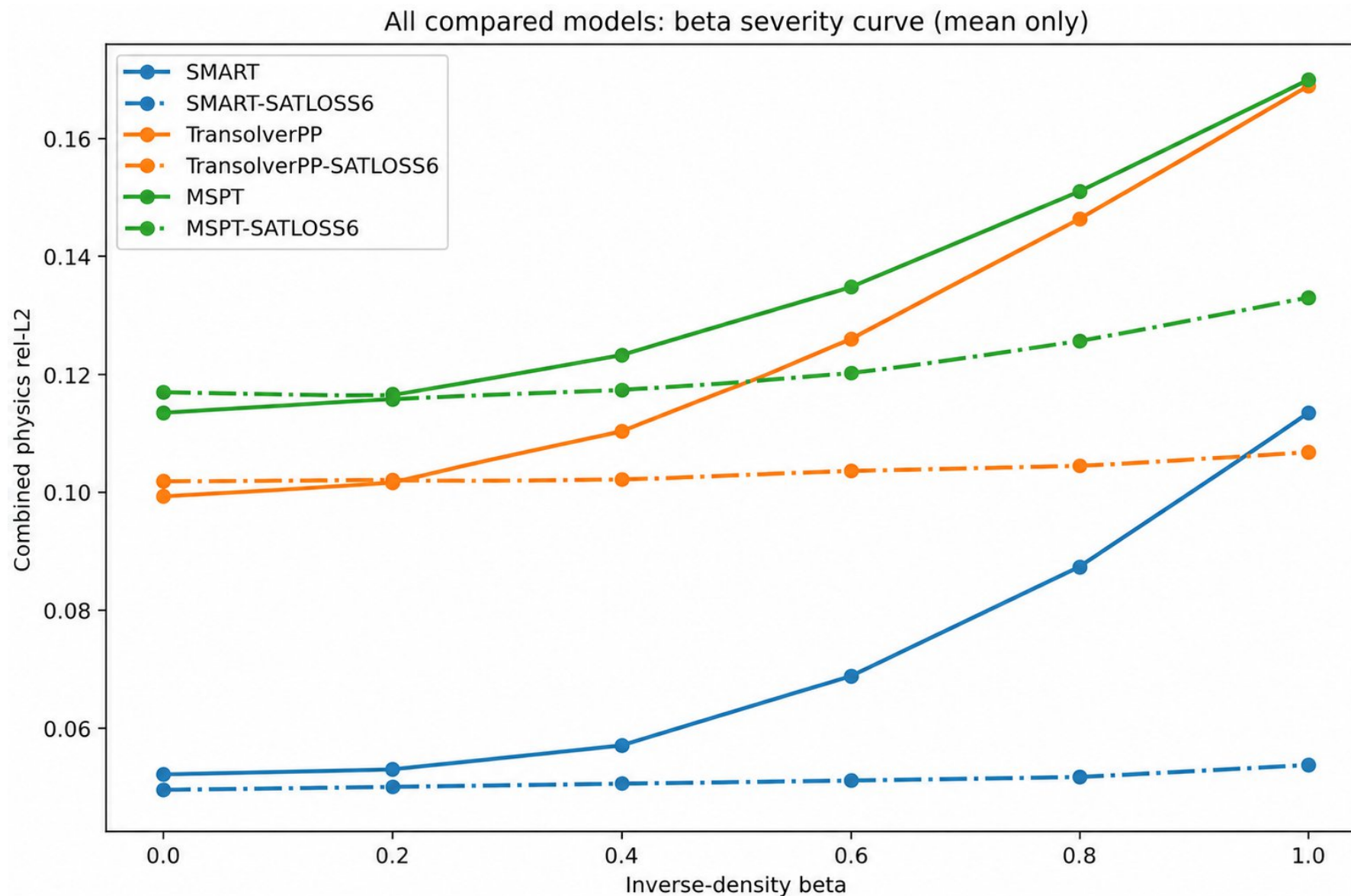
Update with Nils and Parsa



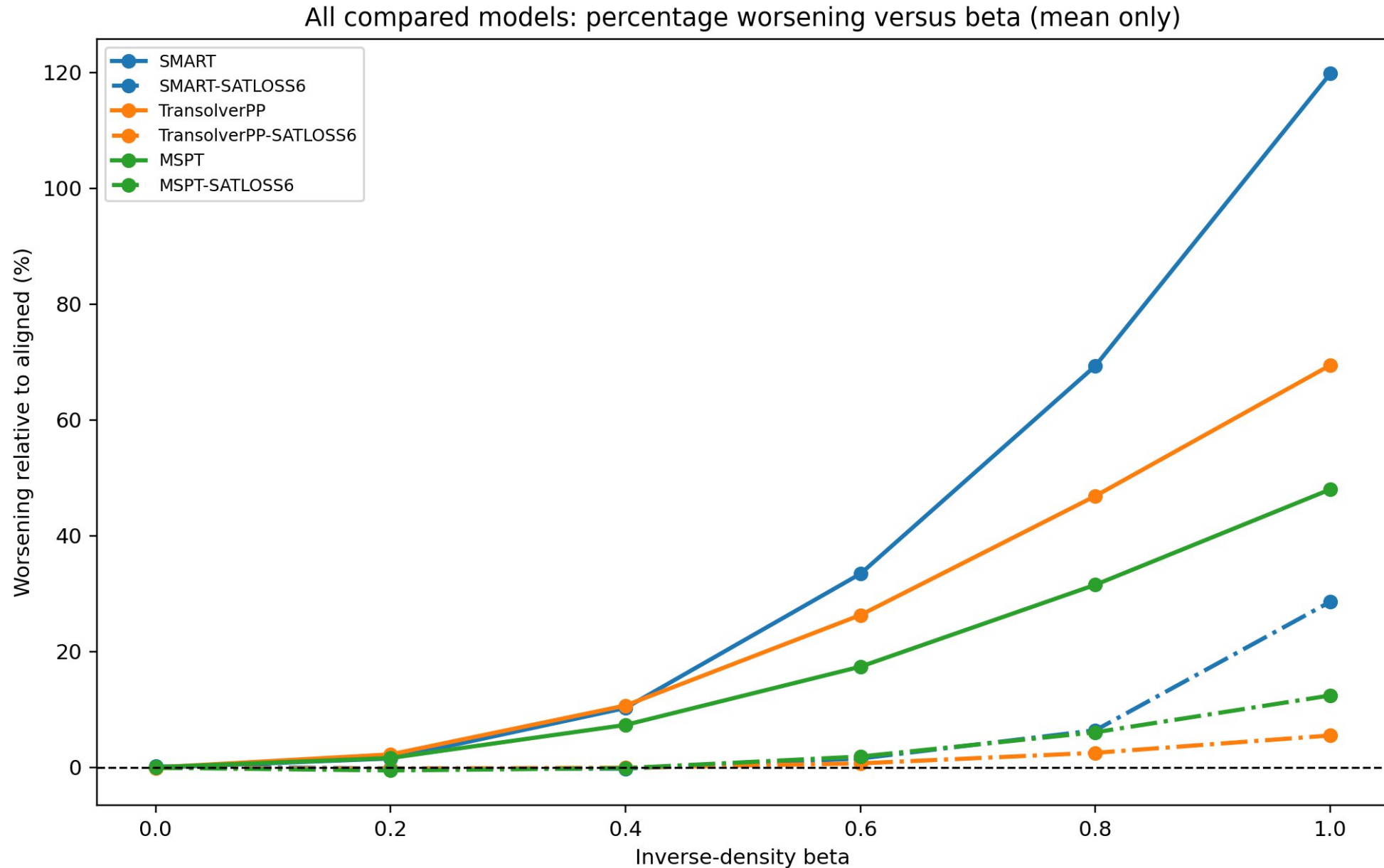
Run 34 sampled input density histogram overlay (beta=0.00 vs 1.00, points=131072)



Trying different models – beta test

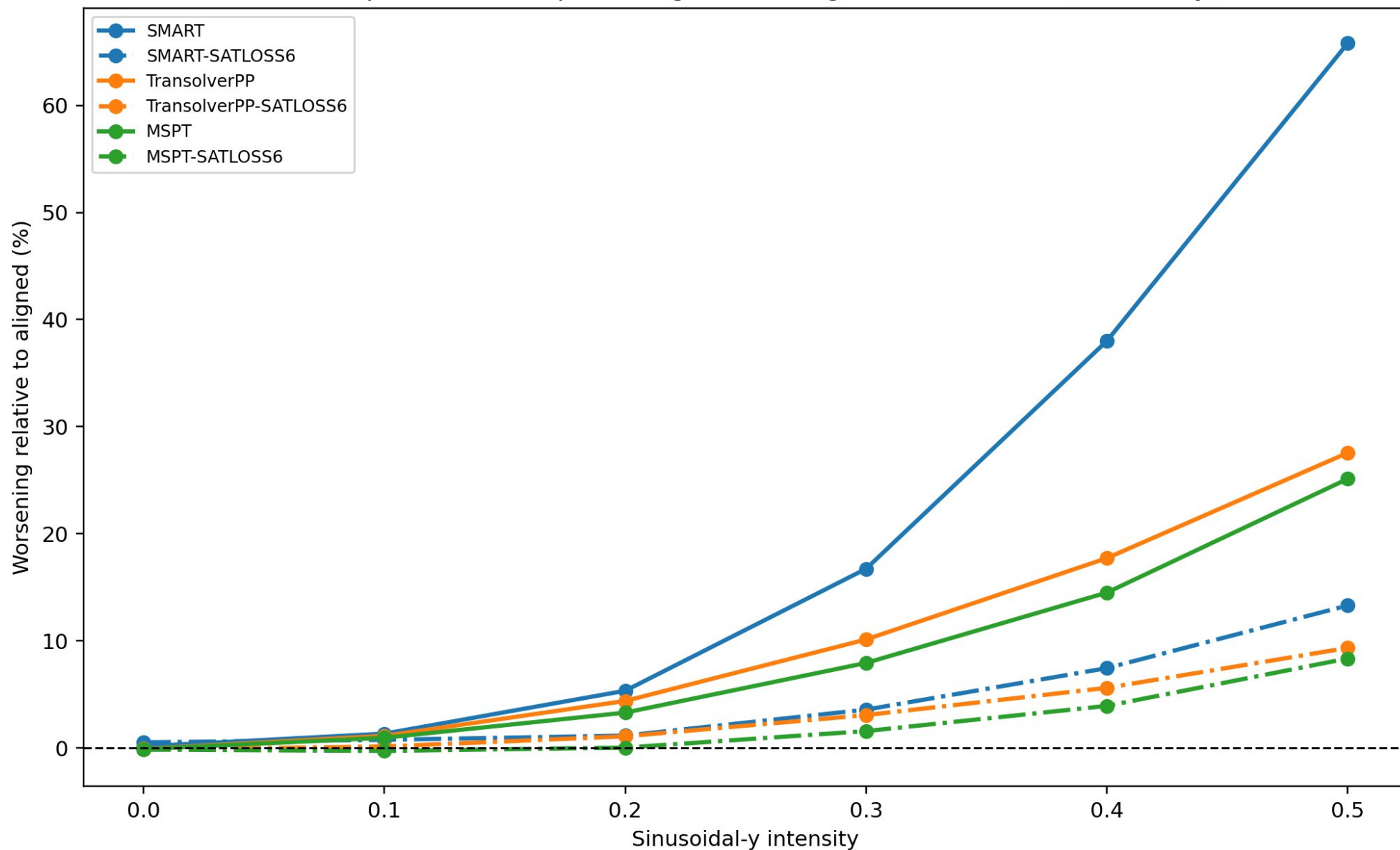


Trying different models – beta test (relative to beta 0)

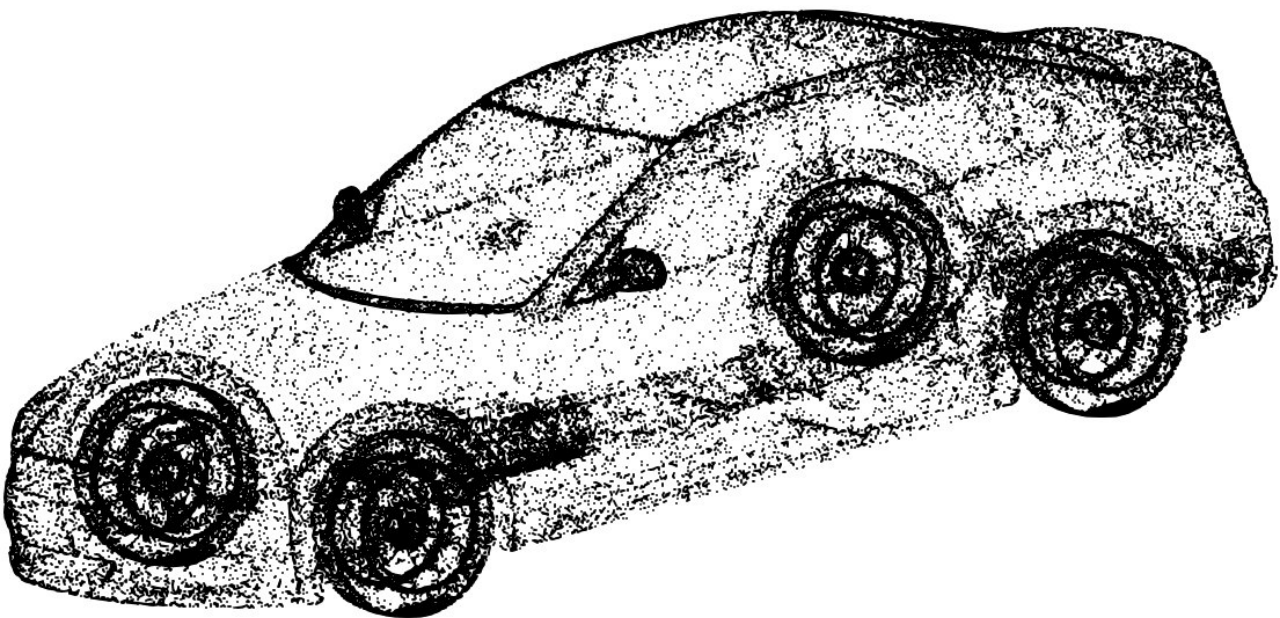


Trying different models – sine test (relative to sine 0)

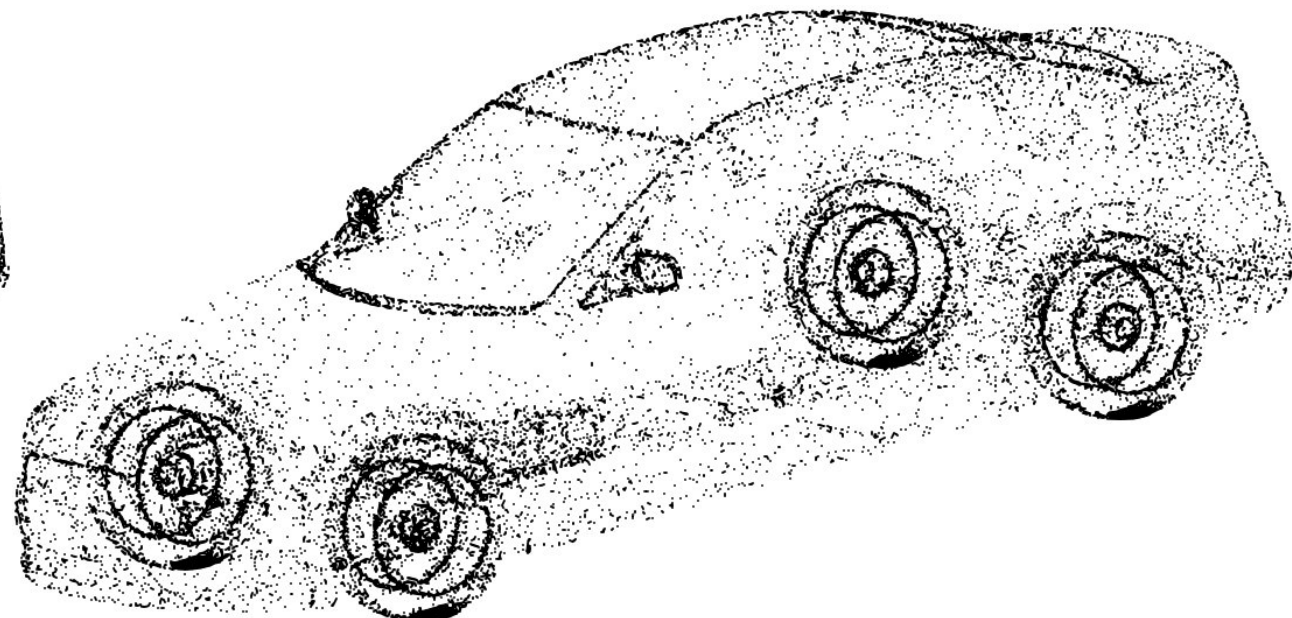
All compared models: percentage worsening versus sine shift (mean only)



Previous methodology in the literature: Random mask



131k points - view 1



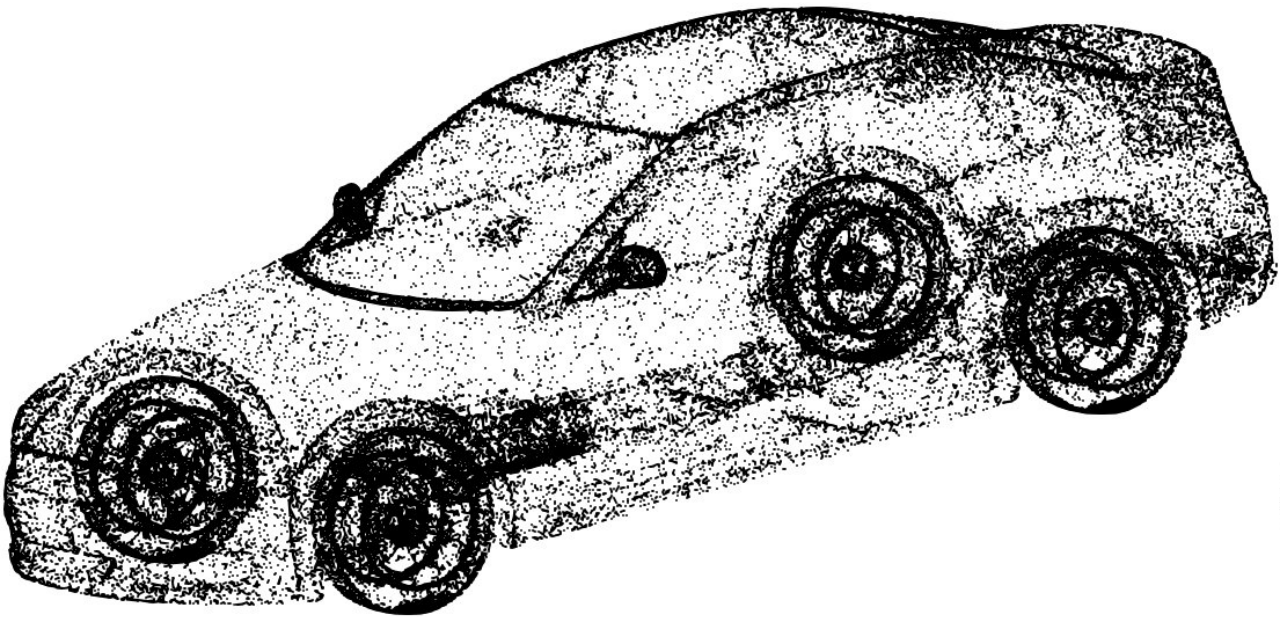
33k points - view 2



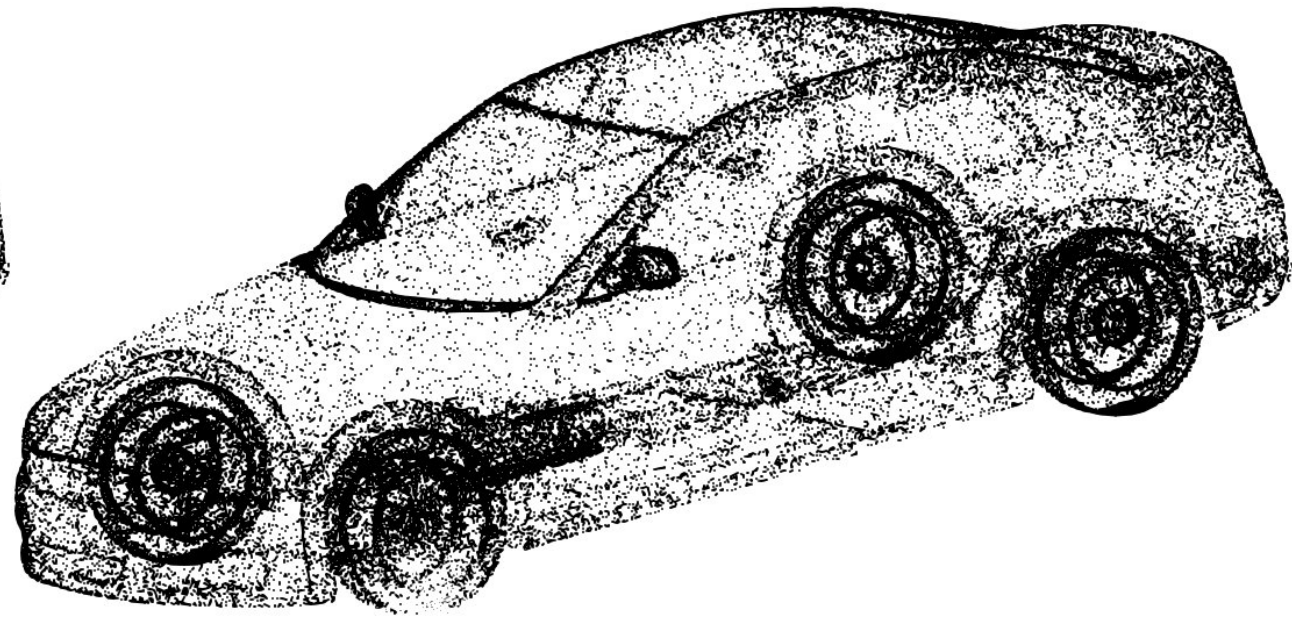
Prediction match loss



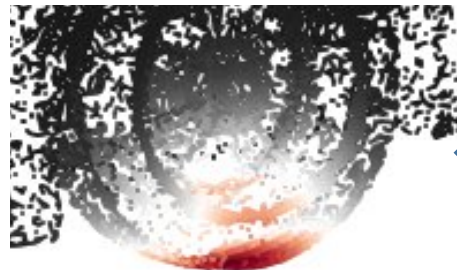
Previous methodology in the literature: Gaussian Ball Mask



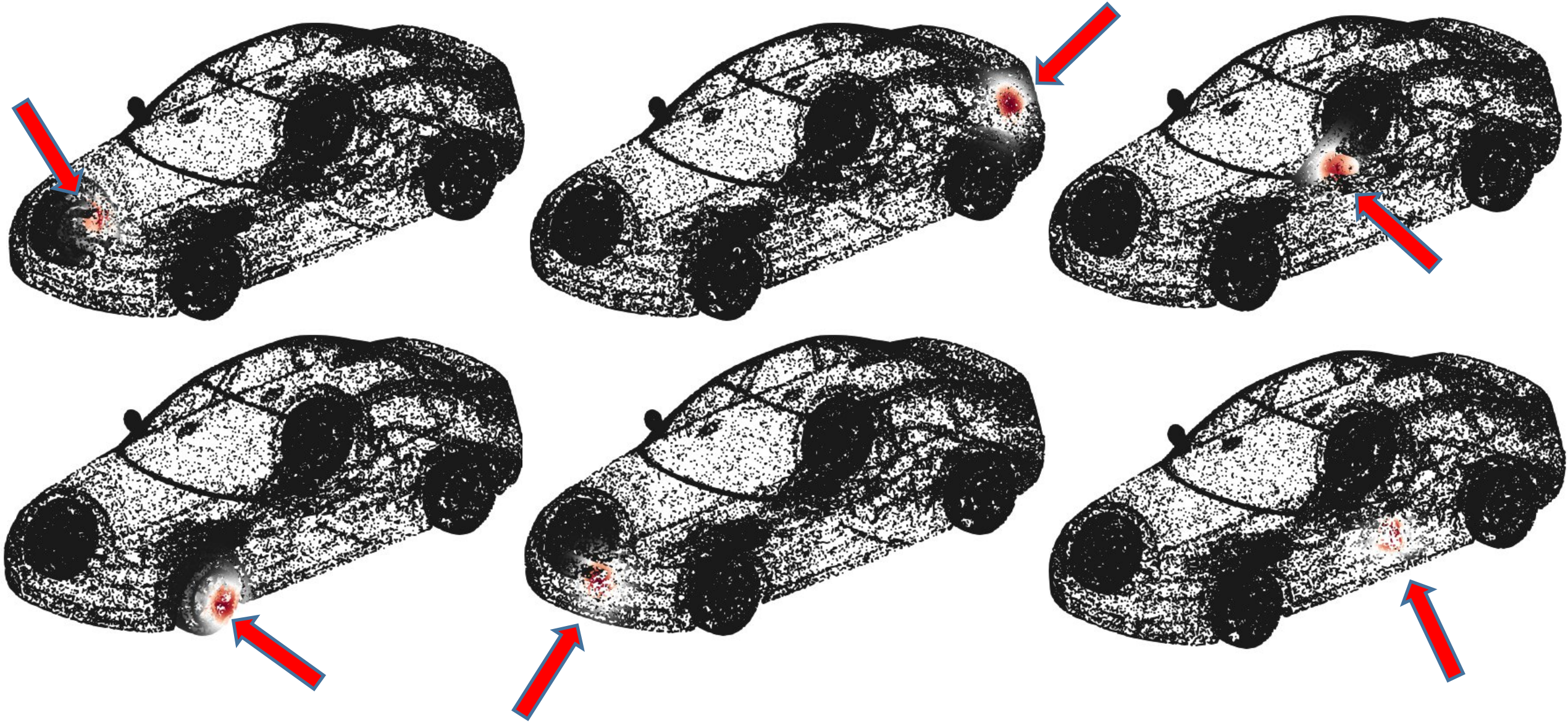
131k points - view 1



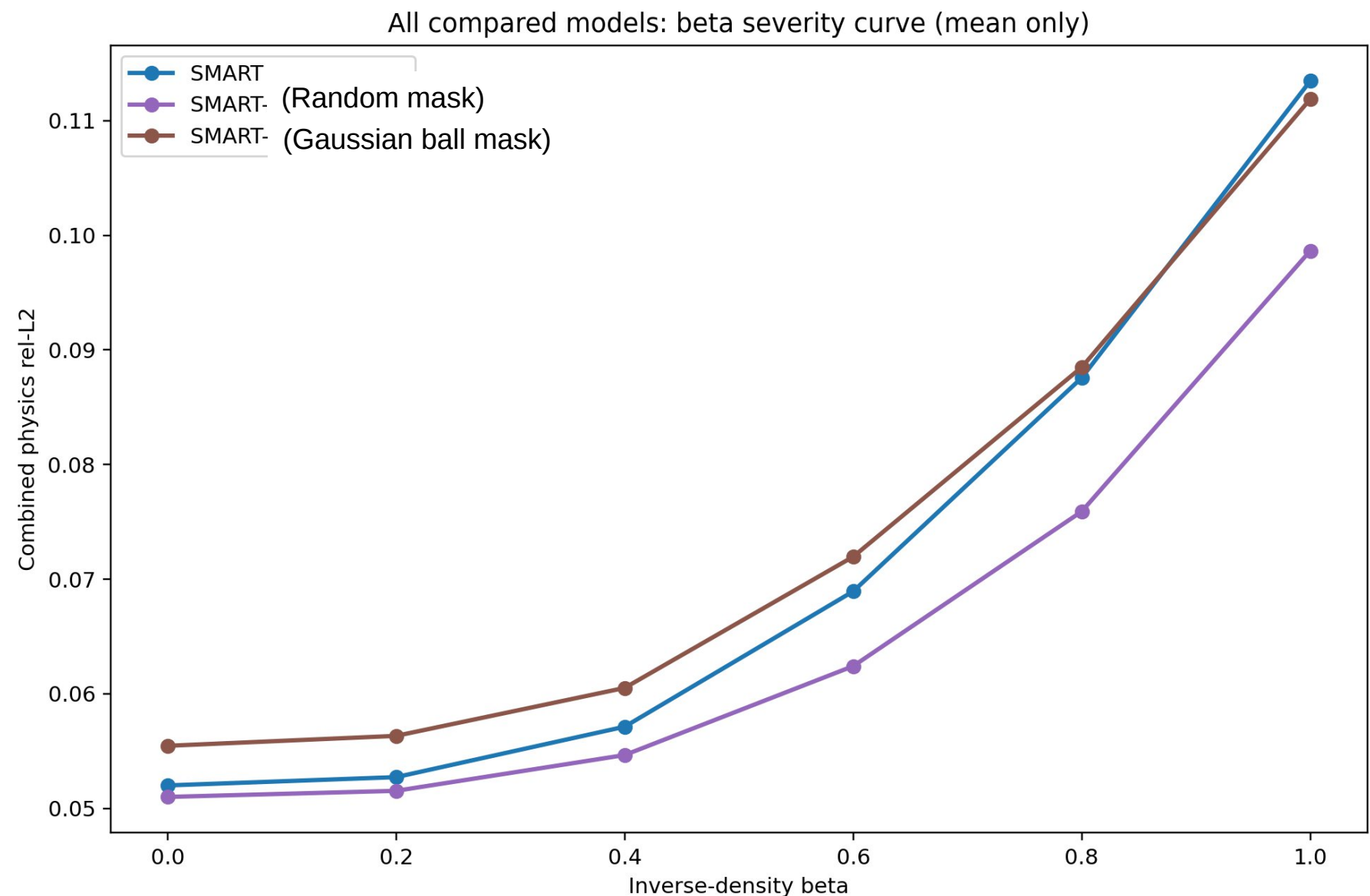
127k points - view 2



Point sampling Baselines – Gaussian Ball Mask

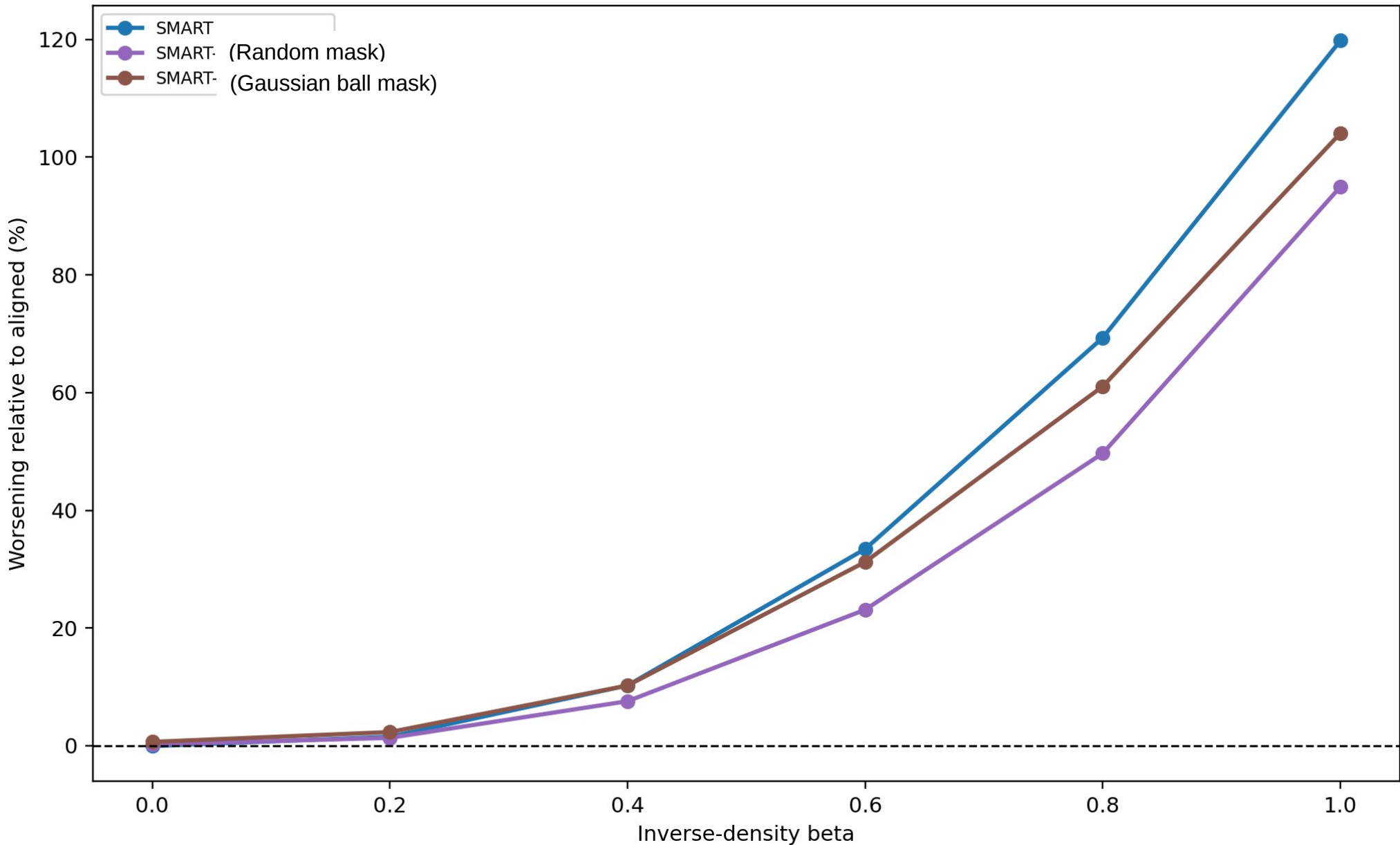


Result of baseline methods - beta



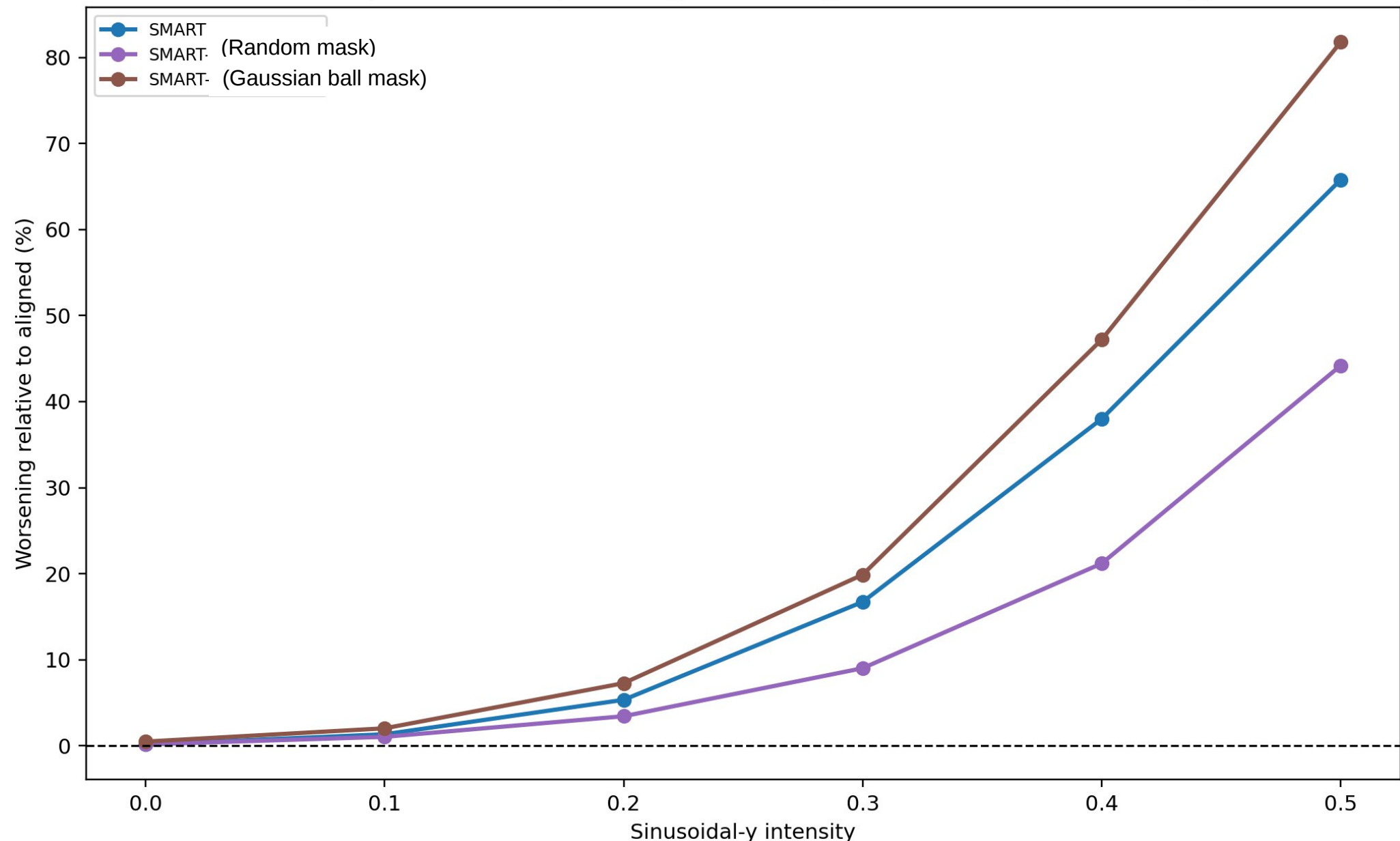
Result of baseline methods- beta (relative to beta 0)

All compared models: percentage worsening versus beta (mean only)

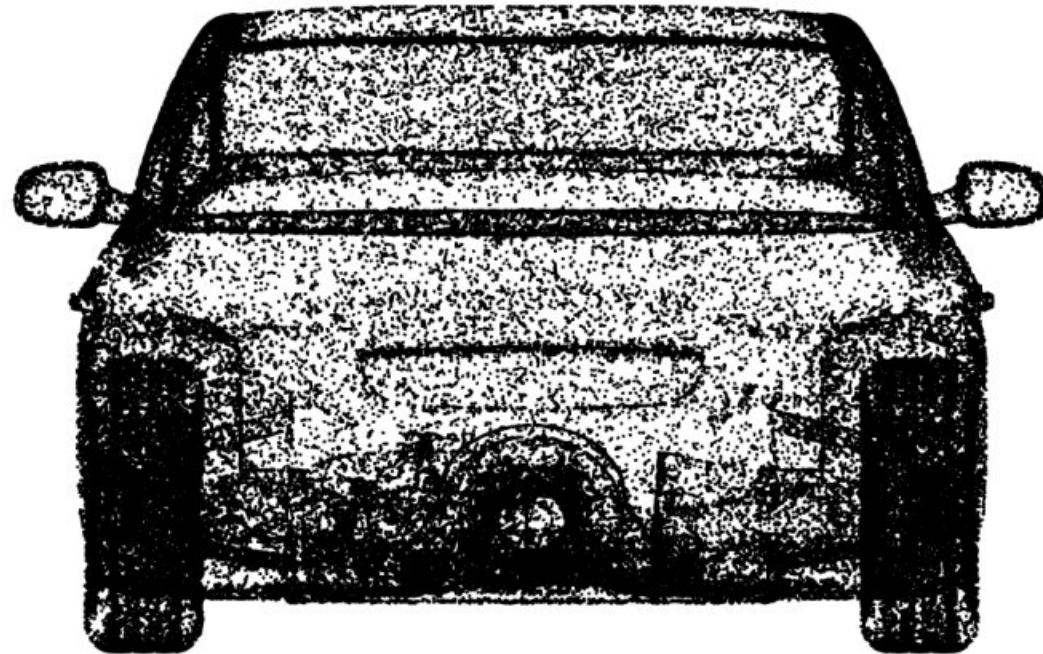
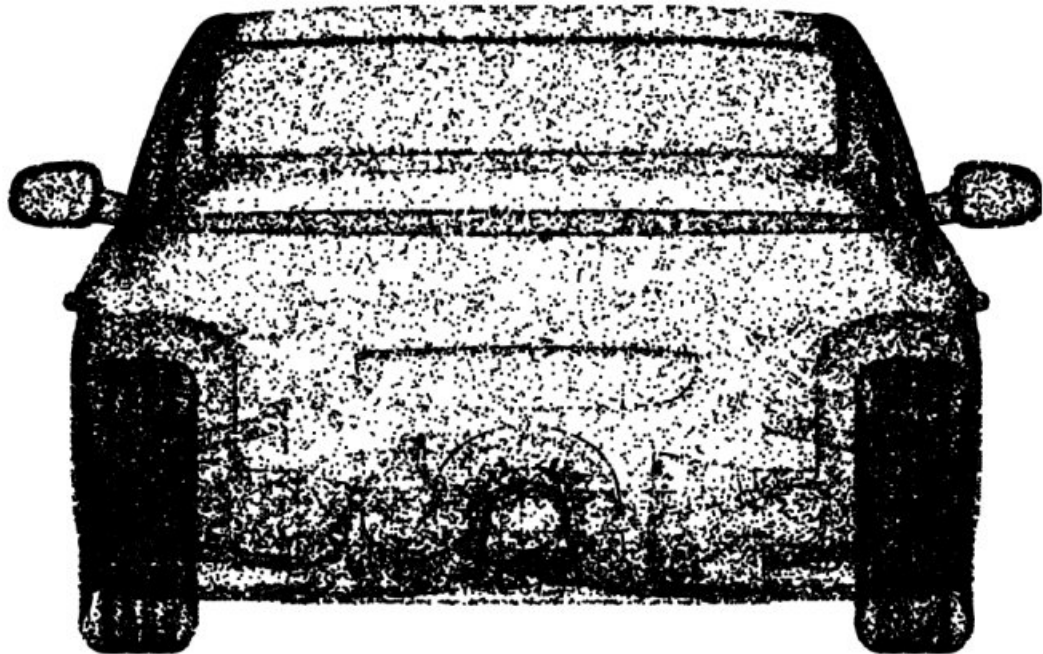


Result of baseline methods- beta (relative to beta 0)

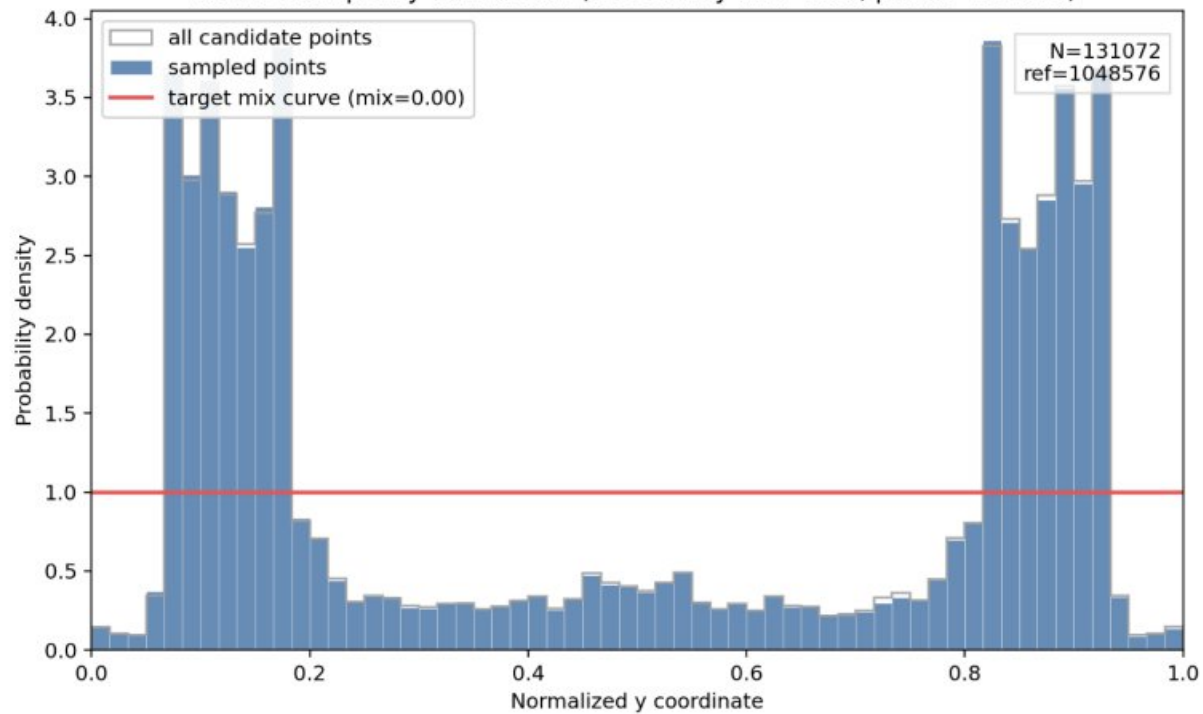
All compared models: percentage worsening versus sine shift (mean only)



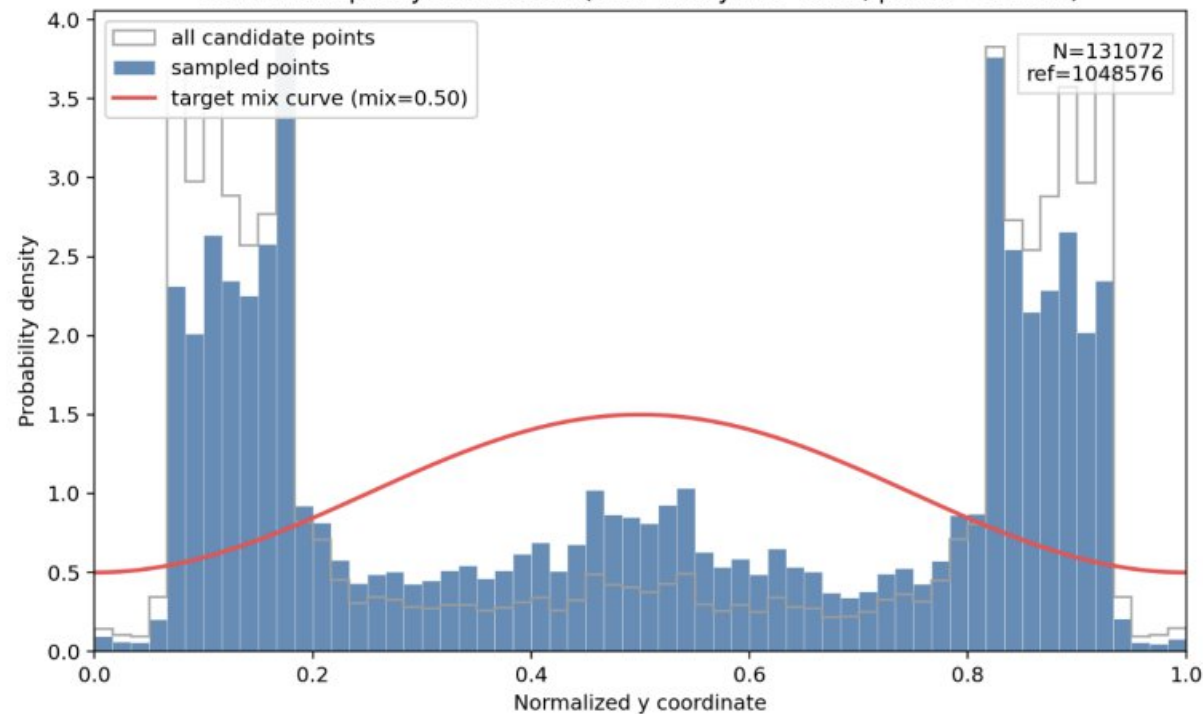
Backup



Run 34 sampled y distribution (OOD sine-y mix=0.00, points=131072)

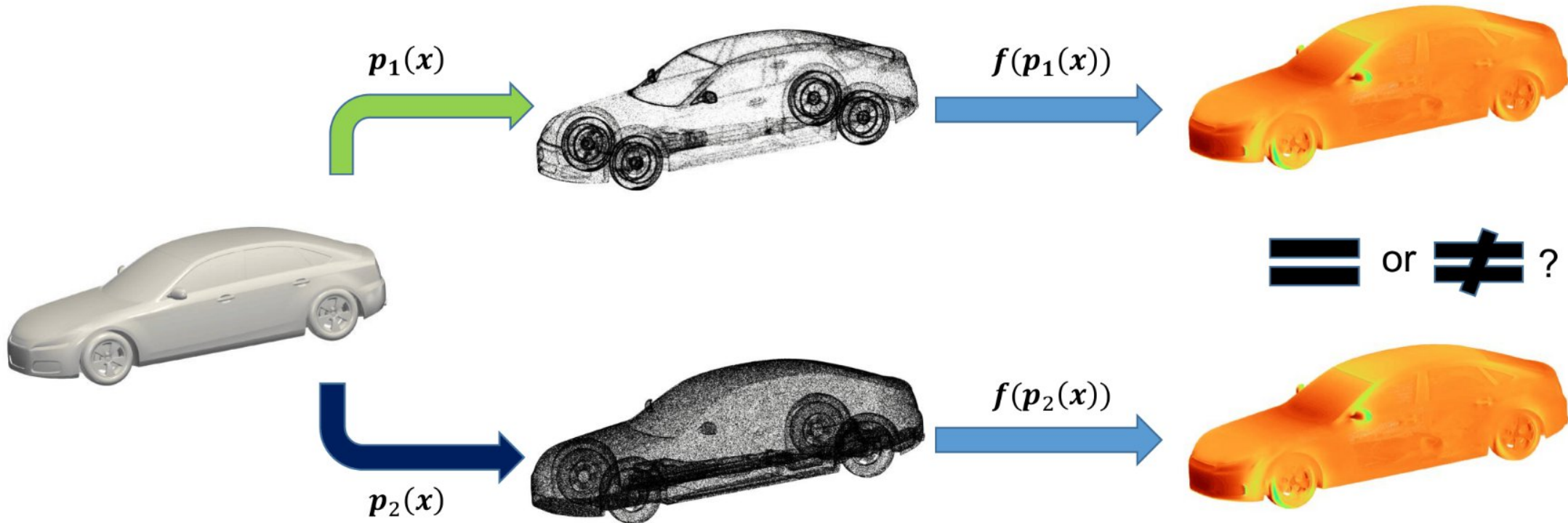


Run 34 sampled y distribution (OOD sine-y mix=0.50, points=131072)

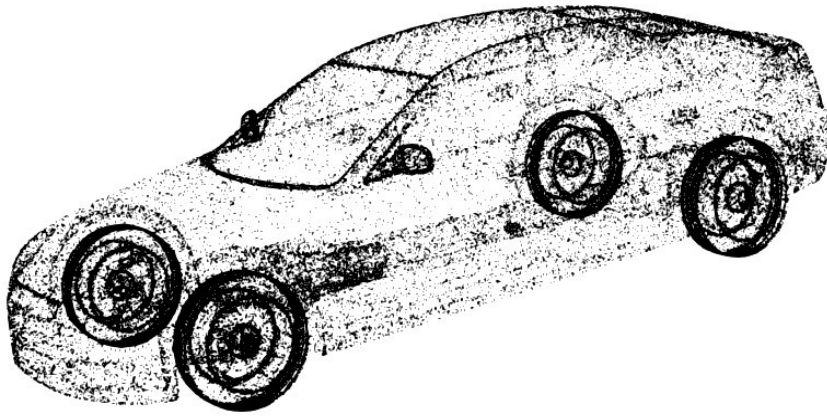


Motivation

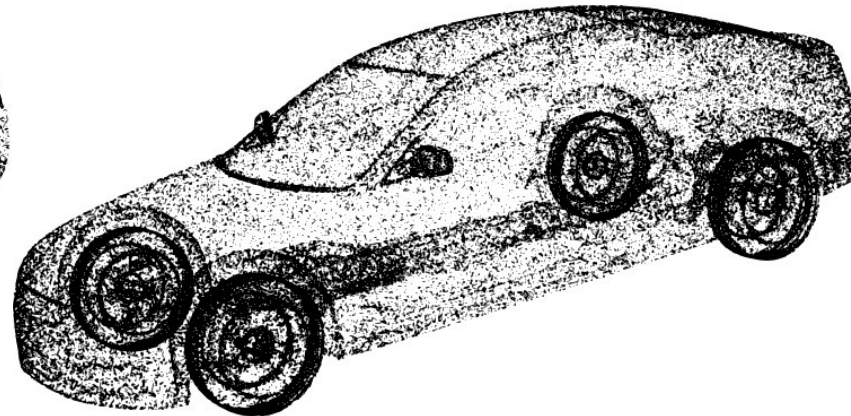
- When we are making prediction y for an input geometry x , our prediction is expected to be $f(x) = y$. However, geometries are first preprocessed into a pointcloud using $p(x)$ and then they are fed to the model. So what we get is actually $f(p(x))$. Not having access to the $p(x)$ poses a challenge for an ideal prediction if the network is not agnostic to how behaves.
- We need to test whether for two different sampling strategy $p_1(x)$ and $p_2(x)$ or not.



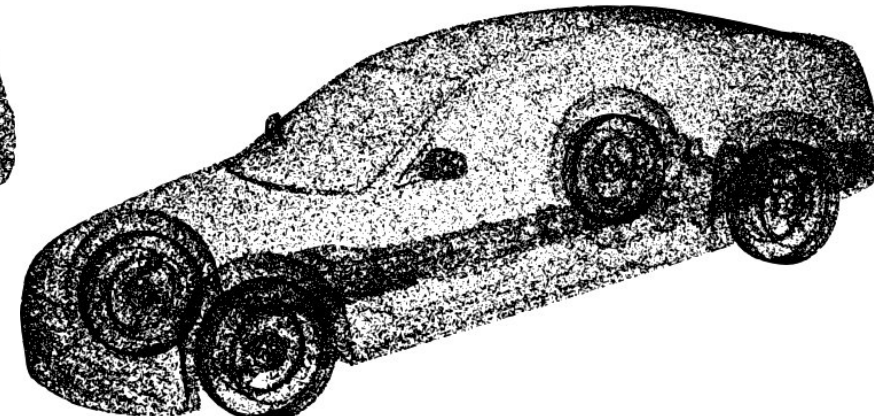
The density-based point sampling



random distribution $\beta=0$



Density correction intensity $\beta=0.5$



Density correction intensity $\beta=1.0$

The algorithm first estimates a **log-density** for each point based on its local k-nearest neighbors (typically $k=24$):

$$\log \rho_j = -\log(d_k^{(j)})$$

Once the log-densities are calculated for all points, the sampling weights and final probabilities are derived:

$$w_i = \rho_i^{-\beta}$$

$$p_i = \frac{w_i}{\sum_j w_j}$$